

# Sisteme de Operare. Laborator 8

Honorius Gâlmeanu  
galmeanu@unitbv.ro

May 18, 2023

## 1 Pipe

Pipe-urile reprezintă cea mai veche modalitate de comunicație între procese (IPC - Inter-Process Communication) și sunt suportate de toate implementările UNIX. Ele au două limitări majore:

1. sunt half-duplex, adică datele pot fi transmise într-o singură direcție;
2. pot fi utilizate numai între procese care au un "strămoș" comun. Un proces va crea un pipe, apoi execută fork, iar pipe-ul va fi utilizat pentru comunicația între procesul părinte și procesul fiu.

În ciuda acestor limitări pipe-urile half duplex reprezintă cea mai utilizată formă de IPC. Un pipe poate fi creat apelând funcția:

```
#include <unistd.h>
```

```
int pipe(int pipefd[2]);
```

Funcția returnează 0 dacă totul este OK și -1 în caz de eroare.

În argumentul pipefd vor fi returnați doi descriptori de fișier: pipefd[0] deschis pentru citire și pipefd[1] deschis pentru scriere. Ieșirea lui pipefd[1] va fi intrare pentru pipefd[0].

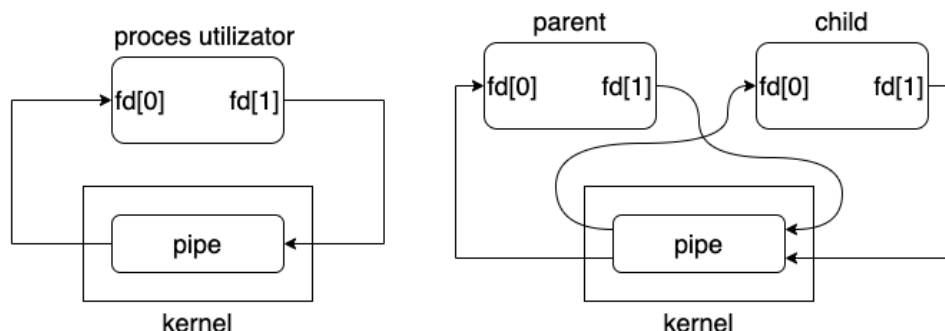


Figure 1: Pipe-ul înainte și după apelul fork.

Folosirea unui pipe pentru un singur proces nu are sens. În mod normal un proces care creează un pipe va apela `fork`, intercalând un canal de comunicație între procesul părinte și procesul fiu. În figura 1 se prezintă situația pipe-ului înainte și după apelul `fork`.

Pentru a realiza un pipe de la părinte către fiu, părintele închide descriptorul deschis pentru citire (`fd[0]`) iar fiul va închide descriptorul deschis pentru scriere (`fd[1]`). Situația aceasta este prezentată în figura 2. Pentru a realiza un pipe de la fiu către părinte, părintele va închide `fd[0]` iar fiul va închide `fd[1]`.

Dacă un capăt al unui pipe a fost închis se aplică următoarele reguli:

1. dacă se citește dintr-un pipe al cărui descriptor de scriere a fost închis, după citirea tuturor datelor, `read` va returna 0 pentru a indica sfârșitul fișierului;

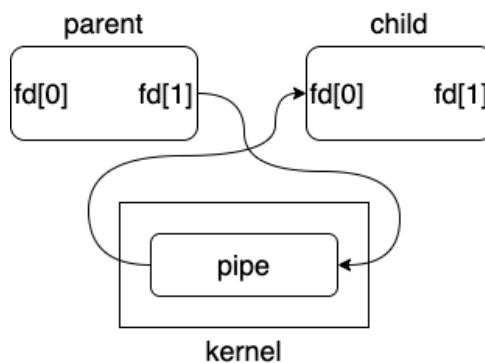


Figure 2: Pipe-ul după închiderea descriptorilor în procesele fiu și părinte.

2. dacă se scrie într-un pipe al cărui descriptor de citire a fost închis, se va genera semnalul SIGPIPE. Dacă acest semnal este ignorat sau tratat și se revine din signal handler, atunci write returnează eroare iar variabila errno va fi setată la valoarea EPIPE.

Dimensiunea bufferului pentru pipe (din kernel) este specificată de constanta PIPE\_BUF. Dacă în pipe vor fi înscrși PIPE\_BUF bytes sau mai puțin, acești bytes nu vor fi întrețesuți cu bytes-ii scrși de alte procese în același pipe. Dacă se depășește dimensiunea dată de PIPE\_BUF atunci datele scrse în pipe de mai multe procese vor fi întrețesute.

## 2 Funcțiile popen și pclose

Funcțiile au următoarele semnături:

```
#include <stdio.h>
```

```
FILE *popen (const char *command, const char *type);
int pclose(FILE *stream);
```

Funcția popen creează un pipe iar apoi execută un apel fork urmat de un apel exec pentru a lansa comanda specificată de cmdstring. Funcția va returna un pointer la un fișier. Dacă argumentul type este "r", pointerul va indica spre ieșirea standard a programului specificat prin command. Dacă argumentul type este "w", pointerul va indica spre intrarea standard a programului specificat prin command.

Funcția pclose închide stream-ul I/O specificat de stream, așteaptă terminarea execuției comenzii executate prin apelul popen și returnează starea terminării execuției. Dacă shell-ul nu se poate executa, starea terminării returnată de pclose este identică cu cea returnată de shell dacă acesta ar fi executat exit(127).

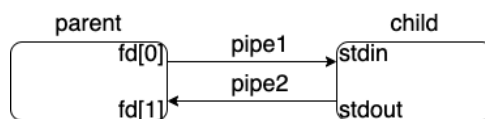


Figure 3: Cum arată un coproces.

Un filtru UNIX este un program care citește de la standard input și scrie la standard output. În mod normal filtrele sunt conectate liniar în pipeline-urile create prin intermediul shell-ului. Un filtru devine coproces când același program îi generează intrarea și îi citește ieșirea (figura 3). Funcția popen permite implementarea coproceselor.

## 3 FIFO

Un pipe putea fi utilizat numai între procese al căror strămoș comun a creat pipe-ul. Prin intermediul FIFO pot comunica și procesele între care nu există relații de precedență (părinte-fiu).

Putem crea un FIFO apelând funcția:

```
#include <sys/stat.h>
int mknod(const char *path, mode_t mode, dev_t dev);
```

Această funcție nu se utilizează doar pentru a crea un FIFO; cu ajutorul ei putem crea și fișiere speciale de tip device. În cazul FIFO argumentul dev va fi ignorat. Argumentul mode va trebui să fie constanta S\_IFIFO.

Odată creat un FIFO, acesta se poate deschide și se pot utiliza toate funcțiile de lucru cu fișiere normale. Ca și în cazul pipe-urilor, se generează semnalul SIGPIPE dacă un proces scrie într-un FIFO și nici un alt proces nu a deschis FIFO pentru citire.

Există două utilizări pentru FIFO:

1. pentru a transmite date între două pipeline-uri fără crea fișiere temporare intermediare;
2. în cazul modelului client-server, pentru comunicația între clienți și server.

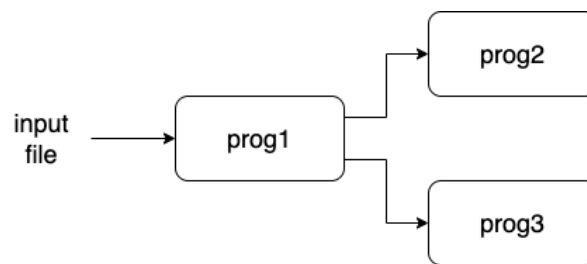


Figure 4: Două procese prelucrează date furnizate de un singur proces.

Vom exemplifica cele două utilizări în continuare. În primul caz să considerăm că două procese vor trebui să prelucreze date de intrare furnizate de un proces filtru (figura 4). Putem realiza acest lucru fără să utilizăm un fișier temporar, ci un FIFO și programul UNIX tee (1) (figura 5).

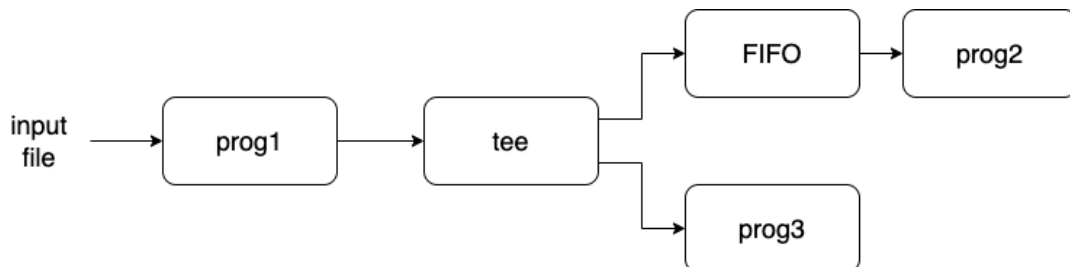


Figure 5: Utilizarea unui FIFO și a programului tee.

Seventă de comenzi va fi:

```
$ mkfifo fifo1
$ prog3 < fifo1 &
$ prog1 < infile | tee fifo1 | prog2
```

A doua utilizare se referă la comunicația între un client și un server. Dacă avem mai multe procese client și un singur proces server, clienții vor partaja un FIFO în care își vor depune cererile. Împreună cu cererea, fiecare client trimite și PID-ul său. Serverul va crea câte un FIFO pentru fiecare client, utilizând pentru formarea numelui FIFO PID-ul fiecărui client. Aceste FIFO-uri vor fi utilizate pentru a trimite răspunsul clientilor (figura 6). De exemplu, serverul va crea un FIFO având numele /temp/serv1.xxxxx, unde xxxxx va fi înlocuit de PID-urile clienților.

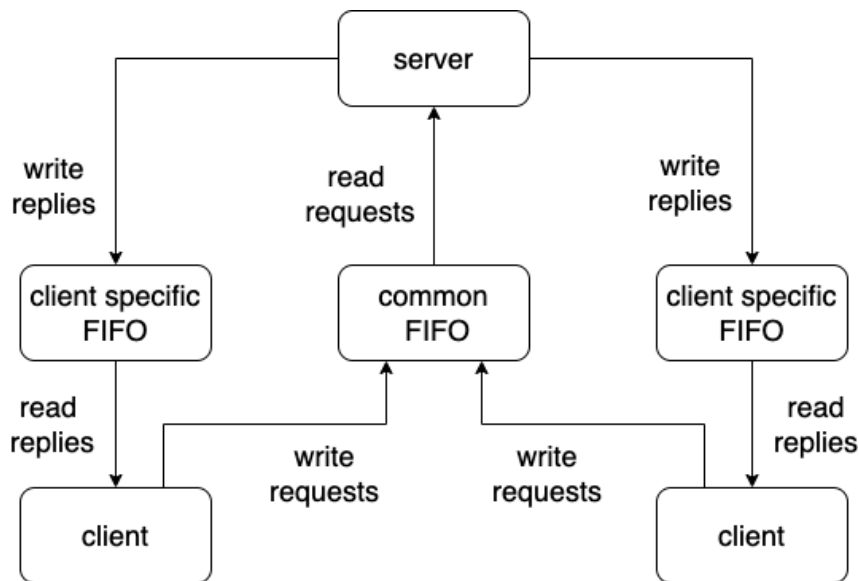


Figure 6: Utilizarea unui FIFO pentru comunicația client-server.

## 4 Example

Pentru a putea compila programele date ca exemplu va fi necesar ca mai întâi să se construiască biblioteca `liblab8.a`. Acesta se face cu comanda:

Command Line

```
make lib
```

După generarea bibliotecii `liblab8.a`, se compilează exemplele cu:

Command Line

```
make
```

### 4.1 Programul `pipe1`

Programul creează un pipe de la procesul părinte la procesul fiu, apoi procesul părinte trimite date prin acest pipe către procesul fiu. Programul se va executa astfel:

```
$ ./pipe1
```

### 4.2 Programul `pipe2`

Programul afisează pagină cu pagină conținutul unui fișier text. În locul rescrierii unui program de paginare, vom utiliza un program de paginare existent (`more`) care va prelua datele prin intermediul unui pipe. În program se creează un pipe, se execută `fork` iar apoi se setează standard input-ul pentru procesul fiu ca fiind capătul prin care este citit pipe-ul și se execută (`exec`) programul de paginare. Programul se va executa astfel:

```
$ ./pipe2 pipe2.c
```

### 4.3 Programul tellwait2

În acest exemplu se dau implementările pentru funcțiile TELL\_WAIT, TELL\_PARENT, TELL\_CHILD, WAIT\_PARENT și WAIT\_CHILD din lucrarea 6. Implementarea acestor funcții a fost realizată utilizând pipe-uri. Se vor crea două pipe-uri înainte de fork (vezi figura 7). Părintele va scrie caracterul "p" în pipe-ul pfd1 atunci când este apelată TELL\_CHILD, iar fiul va scrie caracterul "c" în pipe-ul pfd2 când este apelată TELL\_PARENT. Funcțiile WAIT\_xxx corespunzătoare vor citi printr-un read un singur caracter din pipe.

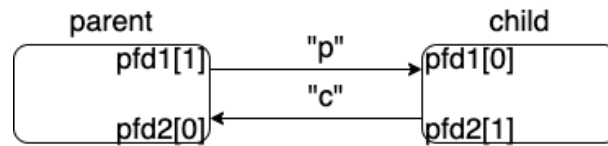


Figure 7: Utilizarea pipe-urilor pentru sincronizarea dintre procesul părinte și procesul fiu.

### 4.4 Programul popen2

Se exemplifică utilizarea funcțiilor popen și pclose. Funcționalitatea programului este identică cu cea a programului pipe2. Se observă că dimensiunea codului se reduce dacă se utilizează popen și pclose. Programul se va executa astfel:

```
$ ./popen2 popen2.c
```

### 4.5 Programul popen1

În acest program se utilizează funcția popen pentru "intercala" un program filtru între programul aplicație și intrarea sa. Programul filtru utilizat (myuc1c) convertește literele mici în litere mari. Programul se va executa astfel:

```
$ ./popen1
```

### 4.6 Programul pipe4

Programul execută coprocesul add2 după ce citește de la standard input două numere. Suma celor două numere realizată de coproces este scrisă la ieșirea standard a procesului părinte. Programul se va executa astfel:

```
$ ./pipe4
```

## 5 Exerciții

#### Exercițiul 1

Rulați și inspectați toate exemplele. Asigurați-vă că ați înțeles cum funcționează.

#### Exercițiul 2

Rescrieți exemplul pipe1 folosind funcția popen. Folosiți în loc de programul pager comanda cat.

### Exercițiul 3

Rescrieți exemplul `pipe4` folosind FIFO. Veți avea două procese: unul care citește din fișier, scrie în FIFO, citește din FIFO și afișează, respectiv un al doilea proces care preia din FIFO parametrii, execută adunarea și scrie rezultatul tot în FIFO.